

Build a Large Language Model from Scratch

this thesis is a summary of [1] and reports on the experience with it.

REPORT 2

Working with text data

Giorgi Gogsadze

Revaz Goguadze

Supervised by Prof. Walter Tichy

Structure

1. What is word embedding
2. The plan
3. Text → Tokens
4. Tokens → IDs
5. Data sampling with a sliding window
6. IDs → Token embeddings
7. Token embeddings → Input embeddings

Except for the exercise solutions, all code and data examples presented in this work are adapted from Chapter 2 of [1].

What is word embedding

The ultimate goal of the report is to generate input embeddings we can feed to LLM, but in order to do so, first we need to understand what is the word embedding itself.

Word embeddings are a way to represent words as numerical vectors in a way that neural networks can process. Since text cannot be directly applied to calculations, embeddings map each word to a continuous vector space. In this space, words with similar meanings or contexts are located near one another. For example, "duck" and "goose" would have similar embeddings as they appear in the same contexts. The simple example is shown in the picture below.

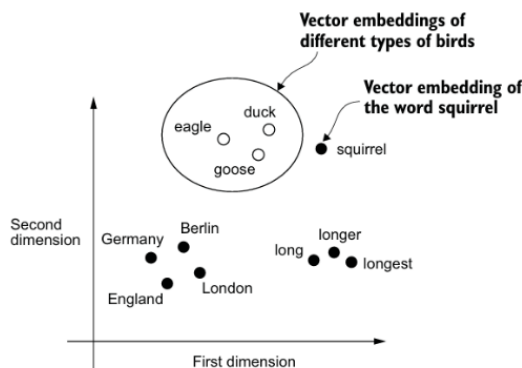


Fig.1: two-dimensional word embeddings (Source: [1], p. 20)

Early methods like **Word2Vec** learned embeddings by predicting the surrounding words of a target word, capturing semantic relationships. In modern large language models (LLMs), embeddings are part of the input layer and are optimized **during training**. This makes them task-specific and more powerful than fixed pretrained embeddings. Of course, The size of embeddings varies, from hundreds to thousands of dimensions, trading off between efficiency (computational cost) and effectiveness (capturing richer meaning).

The plan

In order to fulfill the goal of the report, we need several steps to transform text into embedding vectors that serve as input to a GPT-like decoder-only transformer. The process can be outlined as follows:

1. Tokenization

First, we tokenize the text. This means dividing it into smaller units like words, punctuation characters or special tokens.

2. Mapping to IDs

Next, each token is mapped to an integer ID. This step is essential because neural networks operate on numerical data, not text. We're going to use Byte Pair Encoding (BPE).

3. Dataset preparation

We then construct a dataset and data loader designed for next word prediction. The input tokens are collected into a tensor and a second tensor contains the prediction targets (next word), created by shifting the input sequence by one position.

4. Token embeddings

Since IDs themselves don't capture meaning or context, we map each token ID to a dense vector (embedding). These embeddings are typically initialized randomly and then updated during training.

5. Positional embeddings

Transformers are order-agnostic by design, so we must inject information about word order. This is done using positional embeddings, which are added to the token embeddings.

With all these steps, the input text is converted into meaningful embedding vectors that can be processed by the transformer model. Through the report we're going to use [the verdict](#) as a training data for demonstration.

Text → Tokens

Tokenization is the mechanism for splitting raw text into smaller pieces, which are called tokens. Tokens can be words, subwords, or even individual characters. It is a common preprocessing step when working with language models, since it makes text appear as something that models can operate on.

Let's start implementing all of this and first import the file containing the data.

```
with open("the-verdict.txt", "r", encoding="utf-8") as f:
    raw_text = f.read()
```

Now we need to split this whole document into the list of words and punctuation character.

for example, the sentence:

"The height of his glory"--that was what the women called it.

Should return the following list of tokens:

['', 'The', 'height', 'of', 'his', 'glory', '', '--', 'that', 'was', 'what', 'the', 'women', 'called', 'it', '']

We can achieve this with the help of Python's **re** library:

```
import re
# Split text by whitespace, punctuation, and double dash
preprocessed = re.split(r'([.,:;?_!"()'\]|--|\s)', raw_text)
# Remove empty strings and strip whitespace
preprocessed = [item.strip() for item in preprocessed if item.strip()]
```

The preprocessed list now contains the text's every punctuation mark and word. The models then require special tokens to deal with some context, for example:

- [BOS] beginning of sequence
- [EOS] end of sequence
- [PAD] padding

For our task, [EOS] (which is the same as `<|endoftext|>`) is the most important token. It marks the end of sequences. We can use an `<|unk|>` token optionally since the input could contain words not seen in the training data. But GPT models have Byte Pair Encoding (BPE) tokenizers that break words into subword units, so we need not use `<|unk|>`.

Tokens → IDs

Now, as we have the tokens, it's time to map them to integer IDs. The implementation here is straightforward, take the preprocessed array and remove duplicates. This can be done by converting it into a set. After that sort the set and assign each token an integer corresponding to its position:

```
all_words = sorted(list(set(preprocessed)))
all_words.append("<|endoftext|>") # make sure special tokens are last
vocab = {token:integer for integer,token in enumerate(all_words)}
```

With this, we have a vocabulary containing token-ID pairs:

```
('!', 0)
('\"', 1)
('\"', 2)
...
('Her', 49)
('Hermia', 50)
...
('<|endoftext|>', 1130)
```

Now we need to implement a text tokenizer class with encode and decode methods, which map tokens to IDs and vice versa, like shown in the picture:

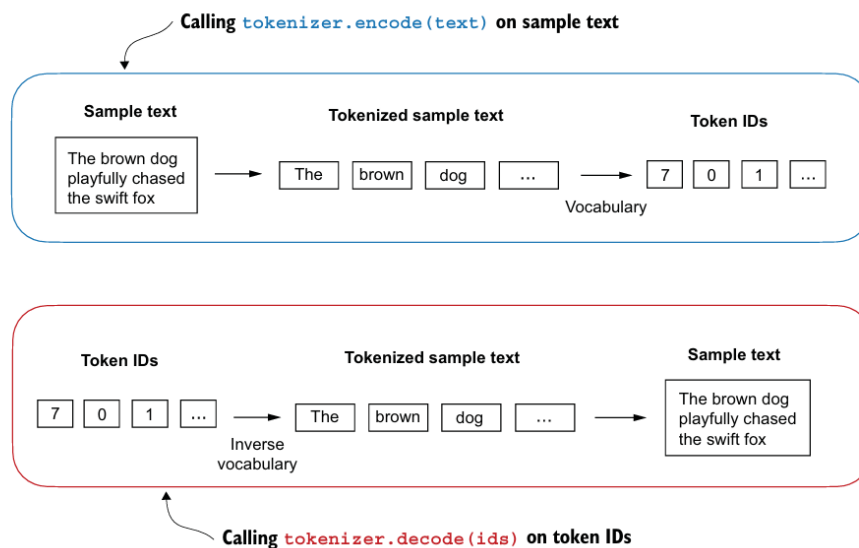


Fig.2: Tokenizer
(Source: [1], p. 28)

Before using BPE, we implement a very simple version of a tokenizer for demonstration purposes:

```
class SimpleTokenizerV1:
    def __init__(self, vocab):
        self.str_to_int = vocab
        # Create an inverse vocabulary that maps
        # token IDs back to the original text tokens
        self.int_to_str = {i:s for s,i in vocab.items()}
    def encode(self, text):
        preprocessed = re.split(r'([,.;?!"()\'`]|--|\s)', text)
        preprocessed = [item.strip() for item in preprocessed if item.strip()]
        ids = [self.str_to_int[s] for s in preprocessed]
        return ids
    def decode(self, ids):
        text = " ".join([self.int_to_str[i] for i in ids])
        # Removes spaces before the specified punctuation
        text = re.sub(r'\s+([,.;?!"()\'`])', r'\1', text)
        return text
```

With simple test we can see that it works well for our text:

```
tokenizer = SimpleTokenizerV1(vocab)
text = """It's the last he painted, you know,"
       Mrs. Gisburn said with pardonable pride."""
ids = tokenizer.encode(text)
print(ids)
```

results in:

```
[1, 56, 2, 850, 988, 602, 533, 746, 5, 1126, 596, 5, 1, 67, 7, 38, 851, 1108, 754, 793, 7]
```

And decoding:

```
print(tokenizer.decode(ids))
```

returns expected result:

```
" It' s the last he painted, you know," Mrs. Gisburn said with pardonable pride.
```

However, at this point there is no specific implementation for unknown words so let's start using BPE.

Use Byte Pair Encoding (BPE)

BPE is a data compression algorithm adapted for text tokenization. Instead of storing entire words, it builds a vocabulary of characters and frequently occurring subwords.

The main idea:

- Start with individual characters.
- Repeatedly merge the most frequent pair of tokens (characters or subwords) into a new token.
- Continue until you reach a target vocabulary size.

This produces a compact vocabulary where common words become single tokens, and rare/unseen words are represented as a sequence of subwords or characters.

BPE is particularly useful because:

1. Common words become single tokens

Example: "the" or "because" don't need to be split.

2. Rare or unseen words are split, no need for <|unk|>

Example:

- "unhappiness" → un, happiness
- "Akwirw" → Ak, w, ir, w

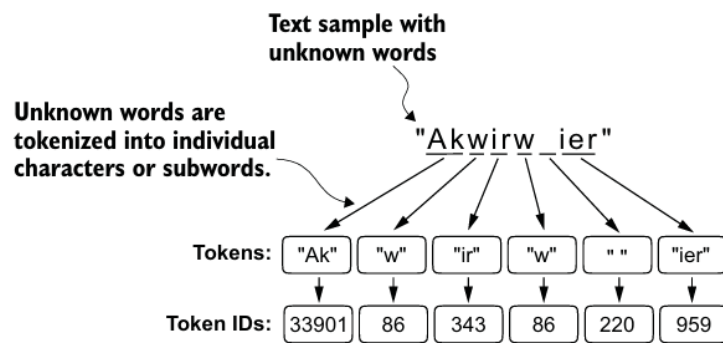


Fig.3: BPE tokenizer visualization
(Source: [1], p. 34)

The implementation of BPE is out of scope for this report, but we will use the tiktoken library.

```
from importlib.metadata import version
import tiktoken

# Load GPT-2 BPE tokenizer
tokenizer = tiktoken.get_encoding("gpt2")

# Sample text
text = (
    "Hello, do you like tea? <|endoftext|> In the sunlit terraces"
    " of someunknownPlace."
)

integers = tokenizer.encode(text, allowed_special={"<|endoftext|>"})
print(integers)
```

This prints all the IDs:

```
[15496, 11, 466, 345, 588, 8887, 30, 220, 50256, 554, 262, 4252, 18250, 8812,
2114, 286, 617, 34680, 27271, 13]
```

To observe it better, let's see what each of these IDs corresponds to:

```
for i in integers:
    strings = tokenizer.decode([i])
    print(strings)

Hello
,
do
you
like
tea
?

<|endoftext|>
In
the
sun
lit
terr
aces
of
some
unknown
Place
.
```

It's clear that some words are divided into subwords, and some of them even have space character merged with it. With decoding we can see that it ideally assembles the input.

```
strings = tokenizer.decode(integers)
print(strings)
```

Hello, do you like tea? <|endoftext|> In the sunlit terraces of someunknownPlace.

Exercise 2.1 also illustrates the logic behind BPE: even words that don't exist in the vocabulary can be represented using subwords and characters.:

```
arr = []
arr.extend(tokenizer.encode("Ak"))
arr.extend(tokenizer.encode("w"))
arr.extend(tokenizer.encode("ir"))
arr.extend(tokenizer.encode("w"))
arr.extend(tokenizer.encode(" "))
arr.extend(tokenizer.encode("ier"))
print(arr)
print(tokenizer.decode(arr))
```

results in:

```
[33901, 86, 343, 86, 220, 959]
Akwirw ier
```

Data sampling with a sliding window

The next step in creating the embeddings for the LLM is to generate the input–target pairs required to teach an LLM how to predict the next word in a sequence. A common and effective method for creating these pairs is the "sliding window" approach.

This approach suggests that we walk through the whole text, take some number of tokens as an input (this amount is called max length or context size) and create a target by shifting the input by one position. Then, jump to other tokens and take them as an input. The number of tokens we jump over is called stride. It's good practice to make stride equal to max length as in this case we utilize the data set fully (we don't skip a single word) and avoid any overlap between the batches, which could otherwise increase overfitting.

The image below shows what our goal is (there are words instead of token IDs for demonstration purposes).

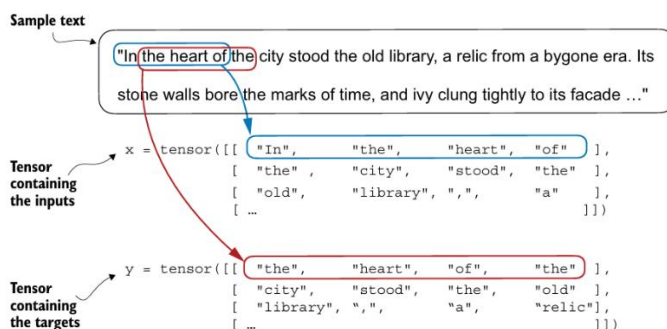


Fig.4: data loader visualization
(Source: [1], p. 37)

Let's start with creating a dataset class that will implement the logic of sliding window approach, as explained above:

```
class GPTDatasetV1(Dataset):
    def __init__(self, txt, tokenizer, max_length, stride):
        self.input_ids = []
        self.target_ids = []

        token_ids = tokenizer.encode(txt)
        for i in range(0, len(token_ids) - max_length, stride):
            input_chunk = token_ids[i:i+max_length]
            target_chunk = token_ids[i+1:i+max_length+1]
            self.input_ids.append(torch.tensor(input_chunk))
            self.target_ids.append(torch.tensor(target_chunk))

    def __len__(self):
        return len(self.input_ids)

    def __getitem__(self, idx):
        return self.input_ids[idx], self.target_ids[idx]
```

and with this class it's easy to create a data loader using PyTorch's DataLoader, that will finally load the inputs in batches and generate input-target pairs.

```
def create_dataloader_v1(txt, batch_size=4, max_length=256, stride=128,
    shuffle=True, drop_last=True, num_workers=0):
    # Initializes the tokenizer
    tokenizer = tiktoken.get_encoding("gpt2")
    # Creates dataset
    dataset = GPTDatasetV1(txt, tokenizer, max_length, stride)
    dataloader = DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=shuffle,
        # drop_last=True drops the last batch if it is shorter than the
        # specified batch_size to prevent loss spikes during training.
        drop_last=drop_last,
        # The number of CPU processes to use for preprocessing
        num_workers=num_workers)

    return dataloader
```

Exercise 2.2 suggests to test data loader and play with its parameters:

```
dataloader = create_dataloader_v1(raw_text, batch_size=2, max_length=2,
    stride=2, shuffle = False)
data_iter = iter(dataloader)
first_batch = next(data_iter)
```

```

print(first_batch)
[ tensor([ [ 40, 367],
           [2885, 1464]]),
  tensor([ [ 367, 2885],
           [1464, 1807]])]

```

Here we can see that the outputs are indeed the inputs shifted by 1, and because `max_length = stride`, the next inputs are direct neighbor tokens.

```

dataloader = create_dataloader_v1(raw_text, batch_size=3, max_length=8,
stride=2, shuffle = False)
data_iter = iter(dataloader)
first_batch = next(data_iter)
print(first_batch)
[ tensor([ [ 40, 367, 2885, 1464, 1807, 3619, 402, 271],
           [2885, 1464, 1807, 3619, 402, 271, 10899, 2138],
           [1807, 3619, 402, 271, 10899, 2138, 257, 7026]]),
  tensor([ [ 367, 2885, 1464, 1807, 3619, 402, 271, 10899],
           [1464, 1807, 3619, 402, 271, 10899, 2138, 257],
           [3619, 402, 271, 10899, 2138, 257, 7026, 15632]])]

```

Here the logic behind the algorithm and parameters is even clearer.

IDs → token embeddings

The final step in text preparation for LLMs is to convert token IDs into embedding vectors. The manual process of mapping token IDs to embedding vectors, described earlier, is automatically implemented by the `torch.nn.Embedding` function. Further explanation is provided in Appendix A. The vectors provide smooth numeric representations of tokens so that language can be learned and processed by neural networks easily.

Embedded layers are initialized with random weights, and the random weights are learned during training through backpropagation. We need 2 parameters here:

1. **vocab_size** specifies the number of token IDs in the vocabulary, and therefore the number of embedding vectors that need to be initialized.
2. **output_dim** defines the dimensionality of each embedding vector. A higher value allows the model to capture more complex relationships between tokens, but also increases computational cost.

In BPE we have a vocabulary of size 50257 and let's say we need 256 dimensional embedding vectors:

```

vocab_size = 50257
output_dim = 256
token_embedding_layer = torch.nn.Embedding(vocab_size, output_dim)
print(token_embedding_layer.weight)

```

Now, we can use this weight matrix to map input IDs to embedding vectors. This is done easily, by mapping the ID to the embedding vector that's present in the weight matrix at the position corresponding to the token ID.

This implementation can be illustrated with a small example (vocab size = 6, embedding dim = 3).

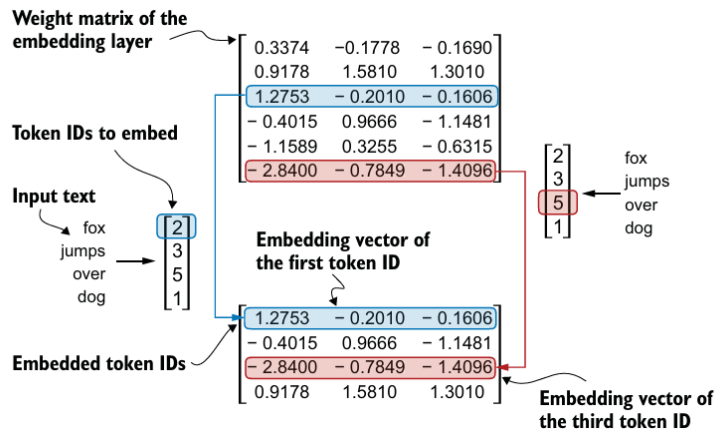


Fig.5: lookup operation
(Source: [1], p. 44)

```
dataloader = create_dataloader_v1(raw_text, batch_size=2, max_length=4,
stride=max_length, shuffle=False)
data_iter = iter(dataloader)
inputs, targets = next(data_iter)
print(inputs)
```

```
tensor([[ 40,  367, 2885, 1464],
        [1807, 3619,  402,  271]])
```

```
token_embeddings = token_embedding_layer(inputs)
print(token_embeddings)
```

```
tensor([[[ 0.4913,  1.1239,  1.4588, ..., -0.3995, -1.8735, -0.1445],
         [ 0.4481,  0.2536, -0.2655, ...,  0.4997, -1.1991, -1.1844],
         [-0.2507, -0.0546,  0.6687, ...,  0.9618,  2.3737, -0.0528],
         [ 0.9457,  0.8657,  1.6191, ..., -0.4544, -0.7460,  0.3483]],

        [[ 1.5460,  1.7368, -0.7848, ..., -0.1004,  0.8584, -0.3421],
         [-1.8622, -0.1914, -0.3812, ...,  1.1220, -0.3496,  0.6091],
         [ 1.9847, -0.6483, -0.1415, ..., -0.3841, -0.9355,  1.4478],
         [ 0.9647,  1.2974, -1.6207, ...,  1.1463,  1.5797,  0.3969]]],
        grad_fn=<EmbeddingBackward0>)
```

Token embeddings → Input embeddings

Self-attention mechanism used in transformers does not inherently capture the order of tokens. Without additional information, the same token ID is always mapped to the same vector, regardless of its position in the sequence. To address this, **positional embeddings** are introduced.

Positional Embeddings can be generated in two main ways:

1. Absolute Positional Embeddings

- Assign a unique embedding to each position in a sequence.
- Added directly to token embeddings, ensuring each token carries both its identity and its location.

2. Relative Positional Embeddings

- Encode the distance between tokens rather than fixed positions.
- Enable the model to generalize better to sequences longer or shorter than those seen during training.

GPT-like models use absolute positional embeddings and that's how we'll implement it. Let's generate positional embedding tensors, which have the same dimension as the token embeddings. We need as many positional embeddings as there are tokens in one input sequence, since we will not need more than that.

```
context_length = max_length
pos_embedding_layer = torch.nn.Embedding(context_length, output_dim)
pos_embeddings = pos_embedding_layer(torch.arange(context_length))
print(pos_embeddings)

tensor([[ 1.6459, -1.2114, -0.9353, ..., -0.0461, -0.9937, -0.0690],
        [ 1.4213, -2.6221, -2.5158, ..., -1.3071, -1.0946, -2.8479],
        [-0.0892,  0.5589,  0.5502, ...,  0.0323, -0.4755, -0.0566],
        [-1.3638, -0.0531, -0.4435, ...,  1.0636, -1.8907, -1.1526]],
        grad_fn=<EmbeddingBackward0>)
```

The final input for the LLM is generated by adding the token embeddings and positional embeddings.

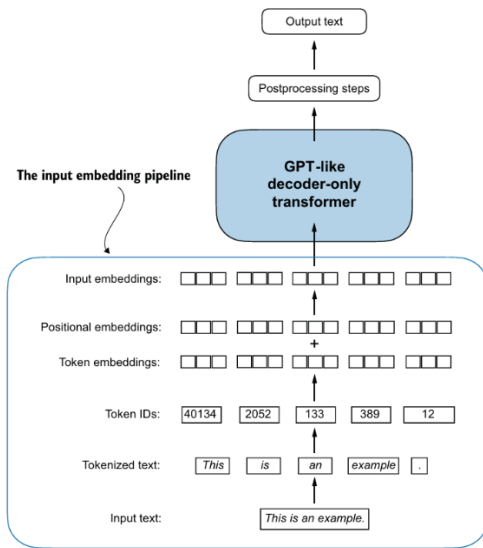


Fig.6: input processing pipeline
(Source: [1], p. 48)

```
input_embeddings = token_embeddings + pos_embeddings
print(input_embeddings)

tensor([[[ 0.0559,  0.1209,  0.8164, ..., -0.9750, -1.0423, -0.8083],
         [ 0.3718,  2.0368, -0.2498, ...,  1.8983, -3.7303, -0.5467],
         [-0.4809, -0.4057,  0.7816, ...,  0.6407,  3.1315,  0.4185],
         [ 0.8596,  0.8846,  1.4093, ..., -0.1684, -0.1808, -0.1241]],

        [[ 1.1107,  0.7339, -1.4272, ..., -0.6759,  1.6896, -1.0058],
         [-1.9384,  1.5918, -0.3655, ...,  2.5206, -2.8807,  1.2468],
         [ 1.7545, -0.9993, -0.0286, ..., -0.7052, -0.1777,  1.9191],
         [ 0.8787,  1.3162, -1.8305, ...,  1.4323,  2.1449, -0.0756]]],
        grad_fn=<AddBackward0>)
```

And now each token embedding is enriched with positional information to create the final **input embeddings** for the LLM.

Appendix A

*“The **torch.nn.Embedding** class in PyTorch provides a simple lookup table that stores embeddings of a fixed dictionary and size. This module is often used to store word embeddings and retrieve them using indices. The input to the module is a list of indices, and the output is the corresponding word embeddings.”*

explanation from [2]

In this report, both token embeddings and positional embeddings are implemented using PyTorch’s `torch.nn.Embedding`. This class provides a lookup table that maps indices to trainable vectors.

For **token embeddings**, each token ID is mapped to a vector of a specified size, forming a weight matrix that is updated during training.

For **positional embeddings**, a similar lookup table is created, where each row corresponds to a position in the input sequence. The input positions are represented as a sequence of indices, and `nn.Embedding` retrieves the vector at each position. these vectors are also learned during training.

By adding token embeddings and positional embeddings, the model receives information about both the identity and the order of tokens.

References

[1] Raschka, S. (2024). *Build a large language model (from scratch)*. Shelter Island, NY: Manning Publications.

[2] PyTorch. (2025). *torch.nn.Embedding* — PyTorch documentation. Retrieved November 13, 2025, from <https://pytorch.org/docs/stable/generated/torch.nn.Embedding.html>